

Analytics tracker in your application. Since the tracker initiation is made inside the *OnInit* function, the tracker will get activated every time the application starts up.

Recording page visits in the single page application

What we need to do is record route-visits.

Now comes the tricky bit – tracking the parts of your application visited by users. From a functional aspect, we know that routes are stand-ins for traditional pages in modern single page Web applications. It means we need to record route-visits. This is not easy, but we can still achieve this.

In the *ngOnInit()* function inside *app.component.ts*, add the following code snippet:

```
import { Router, NavigationEnd } from
 '@angular/router';
...
constructor(public router: Router) {}
...
this.router.events.subscribe(
  event => {
    if (event instanceof
      NavigationEnd) {
      ga('set', 'page', event.
        urlAfterRedirects);
      ga('send', { hitType: 'pageview',
        hitCallback: () => { this.pageViewSent
          = true; }
        });
    }
  } );
```

Believe it or not, those few lines of code have taken care of the 'pageview' recording issue in the Angular application.

Despite only a few lines of code, a fair bit is happening here:

1. Import *Router* and *NavigationEnd* from Angular Router.
2. Add *Router* to the component through its constructor.
3. Next, subscribe to router events; i.e., all events emitted by the Angular Router.
4. Whenever there is an instance of

a *NavigationEnd* event (emitted whenever the application navigates to a route), we set that destination/ route as a page and this sends a pageview.

Now, every time a routing occurs, a pageview is sent to Google Analytics. You can view this in the online Google Analytics dashboard.

Like pageviews, we can record a lot of other activities like screenview, timing, etc, using the same syntax. We can set the program to react in any way we want to for all such *send* activities, through the *hitCallback()*, as shown in the code snippet. Here we are setting a variable value to true, but any piece of code can be executed in *hitCallback*.

Tracking user interactions

After pageviews, the most commonly tracked activities on Google Analytics are user interactions such as, but not limited to, button clicks. 'How many times was the *Submit* button clicked?', 'How often is the product brochure viewed?' These are questions that are often asked in product review meetings for Web applications. In this section, we'll look at this implementation using Google Analytics in the application.

Button clicks: Consider a case for which you wish to track the number of times a certain button/link in the application is clicked – a metric that is most associated with sign-ups, call-to-action buttons, etc. We'll look at an example for this.

For this purpose, assume that you have a 'Show Interest' button in your application for an upcoming event. The organisers wish to keep track of how many people are interested in the event by tracking those clicking the button. The following code snippet facilitates this:

```
...
params = {
  eventCategory:
    'Button'
  ,
  eventAction:
    'Click'
  ,
```

```
  eventLabel:
    'Show interest'
  ,
  eventValue:
    1
};

showInterest() {
  ga('send', 'event', this.params);
}
...
```

Let's look at what is being done here. Google Analytics, as already discussed, records activities when we send the data to it. It is the parameters that we pass to this 'send' method that distinguish between various events like tracking clicks on two separate buttons.

1. First, we define a 'params' object that should have the following fields:
 - a. *eventCategory* – An object with which interaction happens; in this case, a button.
 - b. *eventAction* – The type of interaction; in our case, a click.
 - c. *eventLabel* – An identifier for the interaction. In this case, we could call it *Show Interest*.
 - d. *eventValue* – Basically, the value you wish to associate with each instance of this event. Since this example is measuring the number of people showing interest, we can set this value to 1.
2. After constructing this object, the next part is pretty simple and one of the most commonly used methods as far as Google Analytics tracking goes – sending the event, with the 'params' object as a payload. We do this by using event binding on the button and attaching the *showInterest()* button to it. That's it! Google Analytics (GA) will now track data of people expressing interest by clicking the button.

Tracking social engagements: Google Analytics also lets you track people's interactions on social media through the application. One such case would be a Facebook-type 'Like' button for our brand's page that we place in the